Writeup

HARD

We are given a huge archive containing a release folder. Inside, there are thousands of subdirectories named 0, 1, 2, etc.

We have chal, and .so library file with random name.

running `./chal`, and it asked for an 8-byte input:

Challenge 0: Give me 8 bytes: AAAAAAAA
Wrong!

Okay, classic crackme. But there are more than 1,000 of them!

Decompiling main function in chal returns:

```
 8   v8 = __readfsqword(0x28u);
 9   if ( !dlopen("./RdkQUmBwApnuZltlEDgLE.so", 1) )
10     return 1;
11   buf = 0;
12   printf("Challenge 0: Give me 8 bytes: ");
13   read(0, &buf, 8u);
14   p_buf = &buf;
15   encrypt(&buf);
16   v7 = *p_buf;
17   if ( v7 == 0xAB46C894AA7959EALL )
18     puts("Correct!");
19   else
20     puts("Wrong!");
21   return 0;
22 }
```

int main() { dlopen("./RQzWCuJIedcgSlMkQlBll.so", 1); // Loads the random .so

read(0, &input, 8); // Reads 8 bytes encrypt(&input); // Modifies input in place

if (input == 0xD37BD5CDA83E6217LL) { // Compares with hardcoded hash

puts("Correct!"); }

}

.so library just exported some basic math functions. The real meat was in the `encrypt` function inside the binary itself.

```
17  v14 = __readfsqword(0x28u);
18  v3 = *a1;
19  v4 = a1[1];
20  v5 = 0;
21  for ( i = 0; i <= 255; ++i )
22    v13[i] = i;
23  LOBYTE(v8) = 0;
24  for ( j = 0; j <= 255; ++j )
25  {
26    v8 = (unsigned __int8)(LOBYTE(v13[j]) + v8 + KEY[j % 4]);
27    v12 = v13[j];
28    v13[j] = v13[v8];
29    v13[v8] = v12;
30  }
31  for ( k = 0; k <= 31; ++k )
32  {
33    v10 = v13[v5 & 3] + v5;
34    v1 = log2((double)SHIFTS);
35    v3 += (((v4 >> (int)v1) ^ (v4 << (int)v1)) + v4) ^ v10;
36    v11 = (int)log2((double)dword_4024);
37    v5 += INC_CONST;
38    v4 += (v13[(unsigned __int8)(v5 >> 11)] + v5) ^ (__ROR4__(v3, v11) + v3);
39  }
40  *a1 = v3;
41  a1[1] = v4;
42  return v14 - __readfsqword(0x28u);
43 }
```

It used a few global variables that looked suspicious:

- `KEY` (Array of 4 ints)
- `SHIFTS` (Array of 4 ints)
- `INC_CONST` (1 int)

The logic was:

1. **Key Schedule:** It creates an S-box (0-255) and shuffles it using `KEY` (looks like RC4 logic).
2. **Encryption Loop:** It runs 32 times. In every iteration, it updates the two halves of the input (`first` and `second`) using XORs, additions, and bitwise rotations based on `SHIFTS`

Then we could use apps like bindiff to check for the differences in binaries or check some of them manually.

The code logic was **identical**, but the values in `KEY`, `SHIFTS`, `INC_CONST`, and the final comparison hash in `main` were **different**.

The Plan:

1. Extract the keys (`KEY`, `SHIFTS`, `INC_CONST`) from the data section of every binary.
2. Extract the target hash from the `cmp` instruction in `main`.
3. Implement the inverse of the `encrypt` function.
4. Decrypt all chunks and join them.

We need 'pwntools' to easily parse ELF headers.

The encryption loop adds values to `first` and `second`. To decrypt, we just needed to run the loop backwards (31 down to 0) and **subtract** instead of add.

Note: The encryption rotates bits. We needed helper functions for 32-bit rotations:

def rotr(v, n): return ((v >> n) | (v << (32 - n))) & 0xFFFFFFFF

The target hash was annoying because it's not a variable; it's hardcoded in the assembly: `cmp rdx, 0x...`.

So, we could just use regex on the disassembly output to find the big hex number:

```
asm = elf.disasm(elf.symbols['main'], 1000)
hash_val = int(re.findall(r'movabs .+, (0x[0-9a-f]+)', asm)[-1], 16)
```

After parsing all the values, we just need to reverse the encryption logic and get correct 8 bytes.

At the end we need to add all these bytes and we will get an image with the flag:

The example of a solution script is provided below.